Les fonctions

- Les fonctions
- Passage de paramètres
- La notion de module en C

Rado Rakotonarivo (Cours d'Asma Gabis)
rado.rakotonarivo@efrei.fr

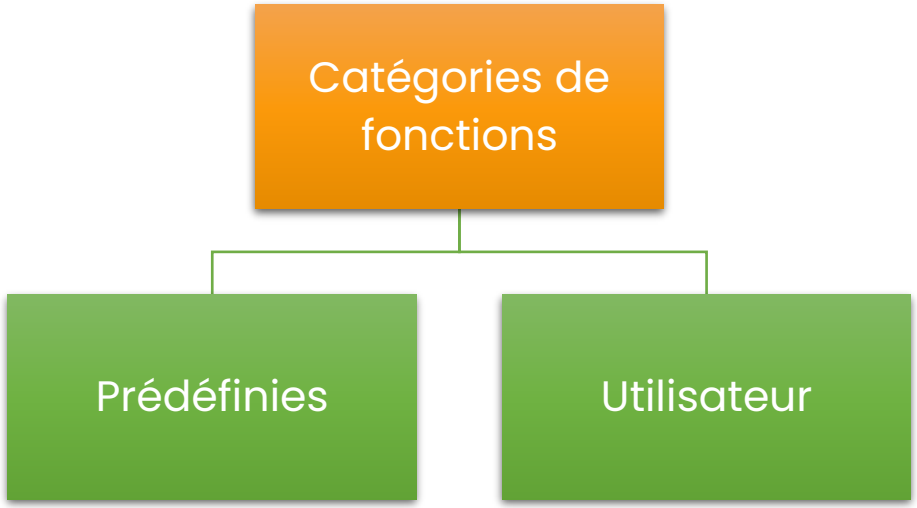
01

Les fonctions

01

Les fonctions

Les catégories de fonctions



Les fonctions

Les fonctions prédéfinies

- Appelées : fonctions de librairies.
- Fonctions prédéfinies dans le langage C
- Plusieurs fonctions prédéfinies ont été déjà utilisées : `printf(), scanf(), malloc(), free(),` …

Comment utiliser une fonction prédéfinie ?

- En incluant son ***fichier.h*** dans le programme.

Attention !! NE METTRE QUE DES .h

#include ‹nom_lib.h›

Mot clé: obligatoire

Obligatoires

Les fonctions

Quelques librairies standards en C

<stdbool.h>

- Bibliothèque standard consacrée aux booléens.
- Contient 4 directives associant **true** à 1 et **false** à 0.
- Ne possède pas de spécificateur printf propre.

```c
#include <stdio.h>
#include <stdbool.h>

int main()
{
    int x;
    bool pair ;
    printf("Saisir un entier : ");
    scanf("%d", &x);

    if (x % 2 == 0)
        pair = true;
    else
        pair = false;

    printf("%d", pair);
    return 0;
}
```

Les fonctions

Quelques librairies standards en C

<string.h>

- Bibliothèque standard consacrée à la manipulation des chaînes de caractères.
- Parmi ses fonctions :
  - **strlen(s1);** retourne la longueur (taille logique) de s1 sans compter le caractère '\0'.

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char mot[32];
    printf("Saisir un mot : ");
    scanf("%s", mot);

    printf("La longueur de %s est : %d.\n", mot, strlen(mot));
    return 0;
}
```

```
Saisir un mot : programmation
La longueur de programmation est : 13.
```

Les fonctions

Quelques librairies standards en C

<string.h>

- Bibliothèque standard consacrée à la manipulation des chaînes de caractères.
- Parmi ses fonctions :
  - **strcpy(s1, s2);** copie s2 dans s1 (dans cet ordre).

```c
#include<stdio.h>
#include<string.h>
int main (){
    char str1[12] = "Hello";
    char str3[12];

    /* copie str1 dans str3 */
    strcpy(str3, str1);
    printf("str1 : %s\n", str1 );
    printf("str3 : %s\n", str3 );

    return 0;
}
```

```
str1 : Hello
str3 : Hello
```

Les fonctions

Quelques librairies standards en C

<string.h>

- Bibliothèque standard consacrée à la manipulation des chaînes de caractères.
- Parmi ses fonctions :
  - **strcat(s1, s2);** concatène s1 et s2 (s2 est copiée à la fin de s1).

```c
#include<stdio.h>
#include<string.h>
int main (){
    char str1[12] = "Hello";
    char str2[12] = "World";

    printf("str1 avant concaténation : %s \n", str1);
    /* concaténe str1 et str2 */
    strcat(str1, str2);
    printf("str1 après concaténation : %s \n", str1);
    /* longueur totale de str1 après concaténation */
    printf("Longueur str1 après concaténation : %d \n", strlen(str1));
    return 0;
}
```

```
str1 avant concaténation : Hello
str1 après concaténation : HelloWorld
Longueur str1 après concaténation : 10
```

Les fonctions

Quelques librairies standards en C

<string.h>

- Bibliothèque standard consacrée à la manipulation des chaînes de caractères.
- Parmi ses fonctions :
  - **strcmp(s1, s2);** retourne 0 si s1 et s2 sont égales; une valeur négative si s1 < s2 (s1 avant s2); une valeur positive si s1 > s2 (s1 après s2).

```c
#include<stdio.h>
#include<string.h>
int main ()
{
    char str1[12] = "Hello";
    char str2[12] = "World";

    printf("Résultat comparaison str1 / str2: %d \n", strcmp(str1, str2));
    printf("Résultat comparaison str2 / str1: %d \n", strcmp(str2, str1));

    return 0;
}
```

```
Résultat comparaison str1 / str2: -15
Résultat comparaison str2 / str1: 15
```

Les fonctions

Quelques librairies standards en C

<math.h>

- Bibliothèque standard permettant d'utiliser un ensemble de fonctions mathématiques de base.
- Parmi ses fonctions :
  - **pow(base, exposant);** retourne le résultat de base puissance exposant.

```c
#include<stdio.h>
#include<math.h>
int main ()
{
    int x =3;
    int y = 2;
    int puissance;

    puissance = pow(x, y);
    printf("%d ^ %d = %d \n", x, y, puissance);
    return 0;
}
```

```
3 ^ 2 = 9
```

Les fonctions

Quelques librairies standards en C

<math.h>

- Bibliothèque standard permettant d'utiliser un ensemble de fonctions mathématiques de base.
- Parmi ses fonctions :
  - **sqrt(n);** retourne la racine carré de n de type float.

```c
#include<stdio.h>
#include<math.h>
int main ()
{
    int x =9;
    int y = 5;

    printf("La racine carré de %d est: %.2f \n", x, sqrt(x));
    printf("La racine carré de %d est: %.2f \n", y, sqrt(y));
    return 0;
}
```

```
La racine carré de 9 est: 3.00
La racine carré de 5 est: 2.24
```

Les fonctions

Quelques librairies standards en C

La fonction rand()

- Ce n'est pas une fonction de la librairie math.h mais elle fait partie de la librairie **<stdlib.h>**
- **rand()** est une fonction renvoie une valeur aléatoirement comprise entre 0 et **RAND_MAX** qui est elle aussi définie dans l'entête <stdlib.h>.
- Le générateur de nombres aléatoires a besoin d'être initalisé via la fonction **srand**

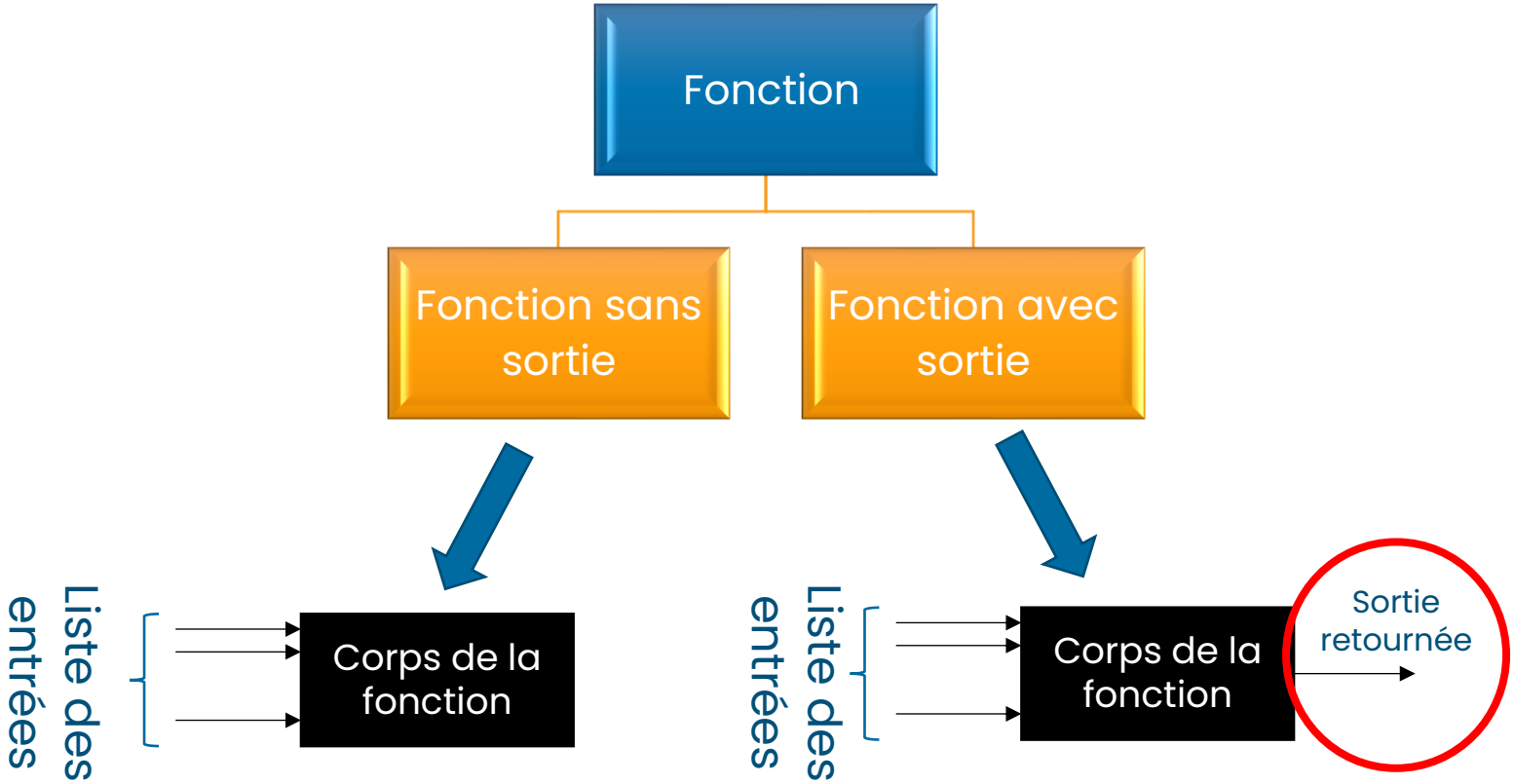
```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int main()
{
    srand(time(NULL)); //
    // rand  renvoie un nombre calculé à partir de la donnée seed
    printf("Nombre aléatoire : %d \n", rand());

    // Pour avoir un nombre entre 0 et 99
    printf("Nombre aléatoire entre 0 et 99 : %d\n", rand() % 100);

    return 0;
}
```

```
Nombre aléatoire : 957999
Nombre aléatoire entre 0 et 99 : 64
```

Les fonctions

Les fonctions utilisateur

Fonctions définies par l'utilisateur durant le développement du programme.



Les fonctions

Caractéristiques d'une fonction

- **Prototype d'une fonction**
  - Appelé aussi: signature
  - Représente la déclaration du sous programme en précisant: son nom, ses paramètres d'entrée et leurs types respectifs, et le type de sortie (s'il existe)
  - Ex: **void somme(int a, int b);**
    int somme (int x, int y);

- **Corps d'une fonction**
  - Est l'ensemble des instructions présentes dans la fonction permettant de réaliser la tâche souhaitée.

Les fonctions

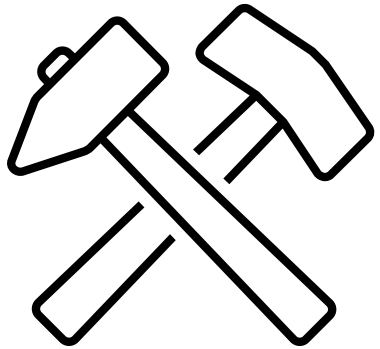
Caractéristiques d'une fonction

- **Entrées**
  - Appelées aussi: paramètres
  - Une fonction peut avoir 0 ou plusieurs entrées
  - Une entrée (si elle existe) représente une variable ➔ {Type nomVariable}

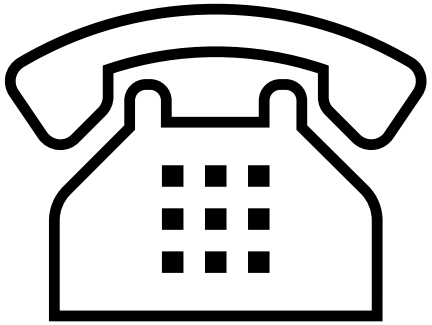
- **Sorties**
  - Une sortie (si elle existe) est représentée **par son type**.

Les fonctions

Utilisation des fonctions en C



Déclaration / définition de la fonction souhaitée



Appel de la fonction au besoin

Les fonctions

Fonctions sans sortie

- **Le mot clé : void**
  - Utilisé pour indiquer une absence d'information sur le type (vide ou nul).
  - beaucoup utilisé avec les fonctions pour indiquer qu'elles ne retournent aucune valeur.
- **Définition d'une fonction sans sortie**

```
void nom_fonction (type_param1 nom_param1, …)

{
    // Définition des variables locales


   // Corps de la fonction
}
```

Les fonctions

Fonctions sans sortie

- **Appel d'une fonction sans sortie**

```
void nom_fonction (type_param1 nom_param1, …)
{
    // Définition des variables locales

   // Corps de la fonction
}
```

Appel

```
type_argument1 nom_argument1;

type_argument2 nom_argument2;

….

nom_fonction (nom_argument1, nom_argument2, …);
```

1. Définition des variables d'entrée: appelés à l'appel « **Arguments** »

2. Les arguments doivent être des mêmes types que les paramètres à la déclaration de la fonction.

3. Appeler la fonction en précisant seulement son nom et la liste des arguments (même nombre dans le même ordre que les paramètres)

Les fonctions

Fonctions sans sortie

- **Appel d'une fonction sans sortie**

```
void nom_fonction (type_param1 nom_param1, …)
{
    // Définition des variables locales

   // Corps de la fonction
}
```

Appel

```
type_argument1 nom_argument1;

type_argument2 nom_argument2;

….

nom_fonction (nom_argument1, nom_argument2, …);
```

MAIS

nom_fonction (**type_Argument1** nom_argument1, **type_argument2** nom_argument2, …)

nom_fonction(nom_argument**2**, nom_argument**1**, …)

Les fonctions

Fonctions sans sortie

- **Exemple 1 : Fonction qui calcule puis affiche la somme de deux entiers**

```
void somme (int a, int b)
{
    int resultat;
    resultat = a + b;
    printf("La somme de %d et %d est: %d \n", a, b, resultat);
}
```

Déclaration de la fonction

```
#include <stdio.h>

int main ()
{
    int x, y;
    printf("Saisir une valeur \n");
    scanf("%d", &x);
    printf("Saisir une valeur \n");
    scanf("%d", &y);
    somme(x, y);
    return 0;
}
```

```
Saisir une valeur
3
Saisir une valeur
13
La somme de 3 et 13 est: 16
```

Appel de la fonction

Les fonctions

Et si le but est de faire le calcul de la somme puis l'utiliser pour une autre opération ?

Les fonctions

Fonctions avec sortie

- **Définition d'une fonction avec sortie**

```
Type_sortie nom_fonction (type_param1 nom_param1, …)

{
      // Définition des variables locales
      Type_sortie resultat;

      // Corps de la fonction
      return resultat;

}
```

Mot clé du langage: **Obligatoire** dans une fonction avec sortie

Les fonctions

Fonctions avec sortie

- **Appel d'une fonction avec sortie**

```
Type_sortie nom_fonction (type_param1 nom_param1,
…)
{
    // Définition des variables locales
   Type_sortie resultat;

   // Corps de la fonction
   return resultat ;
}
```

Appel

```
type_argument1 nom_argument1;

type_argument2 nom_argument2;

Type_sortie res;
….

res = nom_fonction (nom_argument1, nom_argument2, …);
```

1. Définir les variables d'entrée: appelés à l'appel « **Arguments** »
2. Les arguments doivent être des mêmes types que les paramètres à la déclaration de la fonction.
3. Appeler la fonction en précisant seulement son nom et la liste des arguments (même nombre dans le même ordre que les paramètres)
4. Ranger le résultat retourné par la fonction dans une variable du même type que la sortie

Les fonctions

Fonctions avec sortie

- **Appel d'une fonction avec sortie**

```
Type_sortie nom_fonction (type_param1 nom_param1,
…)
{
    // Définition des variables locales
   Type_sortie resultat;

   // Corps de la fonction
   return resultat ;
}
```

Appel

```
type_argument1 nom_argument1;

type_argument2 nom_argument2;

Type_sortie res;
….

res = nom_fonction (nom_argument1, nom_argument2, …);
```

MAIS

nom_fonction(nom_argument1, nom_argument2, …)

res = nom_fonction(**type_argument1** nom_argument1, **type_argument2** nom_argument2, …)

res = nom_fonction(nom_argument**2**, nom_argument**1**, …)

Les fonctions

Fonctions avec sortie

- **Exemple 2 : Fonction qui retourne la somme de deux entiers**

```c
int somme (int a, int b)
{
    int resultat;
    resultat = a + b;
    return resultat
}
```

Déclaration de la fonction

```c
#include <stdio.h>

int main ()
{
    int x, y, res;
    printf("Saisir une valeur \n");
    scanf("%d", &x);
    printf("Saisir une valeur \n");
    scanf("%d", &y);
    res = somme(x, y);
    return 0;
}
```

Appel de la fonction

```
Saisir une valeur
3
Saisir une valeur
13
```

Les fonctions

En général : 4 types de fonctions (définition)

Sans paramètres, sans sortie

```
void nom_fonction()
{
    Corps de la fonction ;
}
```

Avec paramètres, sans sortie

```
void nom_fonction(type1 param1, ....)
{
    Corps de la fonction ;
}
```

Sans paramètres, avec sortie

```
Type_sortie nom_fonction()
{
    Corps de la fonction ;
    return resultat ;
}
```

Avec paramètres, avec sortie

```
Type_sortie nom_fonction(type1 param1, ..)
{
    Corps de la fonction ;
    return resultat ;
}
```

Les fonctions

En général : 4 types de fonctions (appel)

Sans paramètres, sans sortie	Avec paramètres, sans sortie
nom_fonction()	nom_fonction(arg1, ...)
Sans paramètres, avec sortie	**Avec paramètres, avec sortie**
Type_sortie resultat ; **resultat** = nom_fonction()	**Type_sortie resultat** ; **resultat** = nom_fonction(arg1, ..)

Les fonctions

Variables locales Vs variables globales

- **Variable locales**

  - Les variables déclarées à l'intérieur d'une fonction ou d'un bloc sont appelées **variables locales**.

  - Elles ne peuvent être utilisées que par des instructions contenues dans cette fonction ou ce bloc de code.

- **Exemple**

```c
#include <stdio.h>
int carre(int N)
{
    int C;
    C = N * N;
    return C;
}


int main ()
{
    int N, C;
    printf("Saisir une valeur \n");
    scanf("%d", &N);
    C = carre(N);
    printf("%d ^ 2 = %d \n", N, C);
    return 0;
}
```

Ici N et C sont **locales** à la fonction carre() et ne sont pas visibles par la fonction main()

Ici N et C sont **locales** à la fonction main() et ne sont pas visibles par la fonction carre()

Les fonctions

Variables locales Vs variables globales

- **Variable globales**

  - Elles sont définies en dehors d'une fonction, généralement en haut du programme.
  - Les variables **globales** conservent leurs valeurs tout au long de la vie d'un programme.
  - Elles sont accessibles dans n'importe quelle fonction définie dans le programme.

- **Exemple**

```c
#include <stdio.h>

int N;

int carre()
{
    int C;
    C = N * N;
    return C;
}
int main ()
{
    int C;
    printf("Saisir une valeur \n");
    scanf("%d", &N);
    C = carre(N);
    printf("%d ^ 2 = %d \n", N, C);
    return 0;
}
```

Pas besoin de re déclarer N dans la fonction carre()

Pas besoin de re déclarer N dans la fonction main()

02

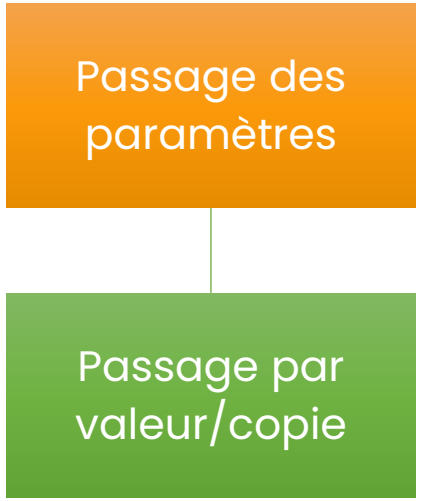
Passage des paramètres

02

Passage des paramètres

Définition

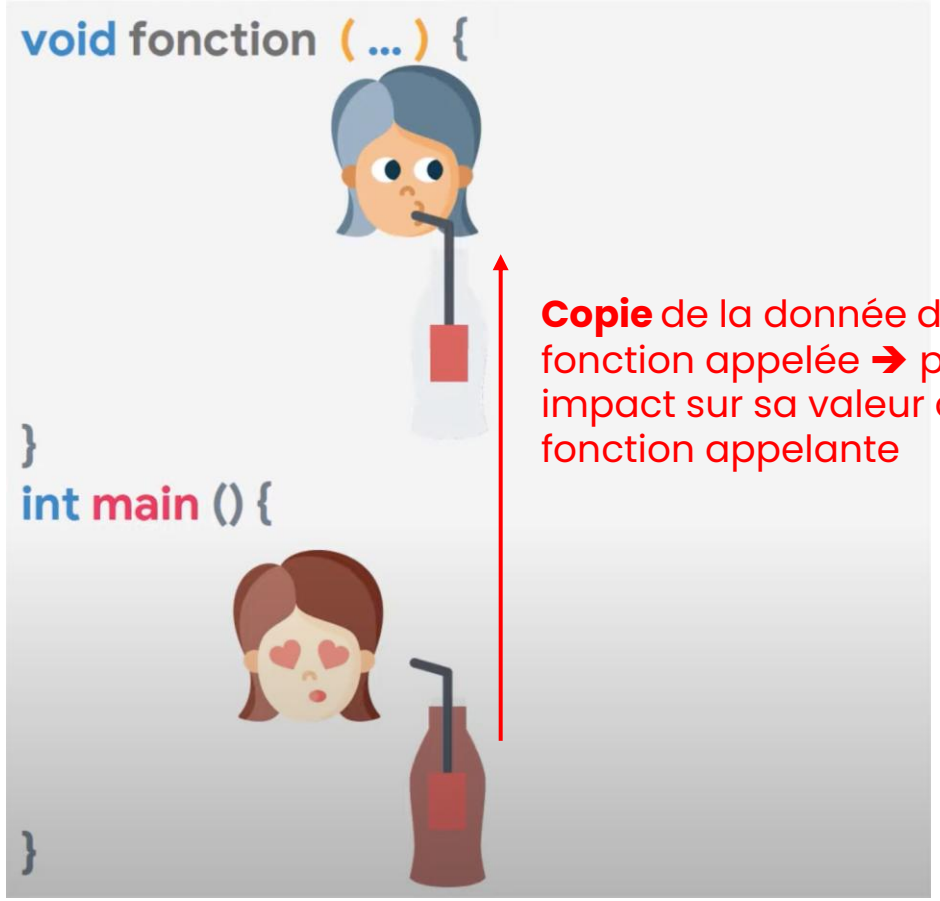
- La transmission d'informations d'un programme appelant vers un programme appelé, se fait à travers les paramètres.
- Le langage C **permet uniquement le passage par valeur, ou un passage par copie.** Les valeurs des paramètres sont copiées dans les variables locales de la fonction.



Passage des paramètres

Passage par valeur/par copie

- Les valeurs des données du programme appelant **sont copiées** dans les paramètres de la fonction appelée.
- les deux données sont stockées dans **des emplacements mémoire différents**.
- Toutes les modifications apportées dans les fonctions appelées **ne sont pas reflétées** sur les données du programme appelant.



Copie de la donnée dans la fonction appelée ➔ pas impact sur sa valeur dans la fonction appelante

Passage des paramètres

Passage par valeur/par copie

Exemple 1

```c
#include <stdio.h>
int fct1(int x)
{
    x = x * 2;
    printf("Valeur de x dans la fonction : x = %d\n", x);
    return x;
}
int main ()
{
    int x, result;
    printf("Saisir une valeur : ");
    scanf("%d", &x);
    printf("Avant l'appel de la fonction : x = %d\n", x);
    result = fct1(x);
    printf("Après l'appel de la fonction : x = %d\n", x);
    printf("Après l'appel de la fonction : result = %d\n", result);
    return 0;
}
```

```
Saisir une valeur : 3
Avant l'appel de la fonction : x = 3
Valeur de x dans la fonction : x = 6
Après l'appel de la fonction : x = 3
Après l'appel de la fonction : result = 6
```

La valeur de **x** n'a pas changé mais le résultat a été retournée de fct1 à main et a été récupéré dans la variable **result**

Passage des paramètres

Passage par valeur/par copie

Exemple 2

```c
#include <stdio.h>
void permutation(int x, int y)
{
    int temp;
    temp = x;
    x = y;
    y = temp;
}
int main ()
{
    int x, y;
    printf("Saisir une valeur : ");
    scanf("%d", &x);
    printf("Saisir une autre valeur : ");
    scanf("%d", &y);
    printf("Avant l'appel de la fonction : x = %d, y = %d\n", x, y);
    permutation(x, y);
    printf("Après l'appel de la fonction : x = %d, y = %d\n", x, y);
    return 0;
}
```

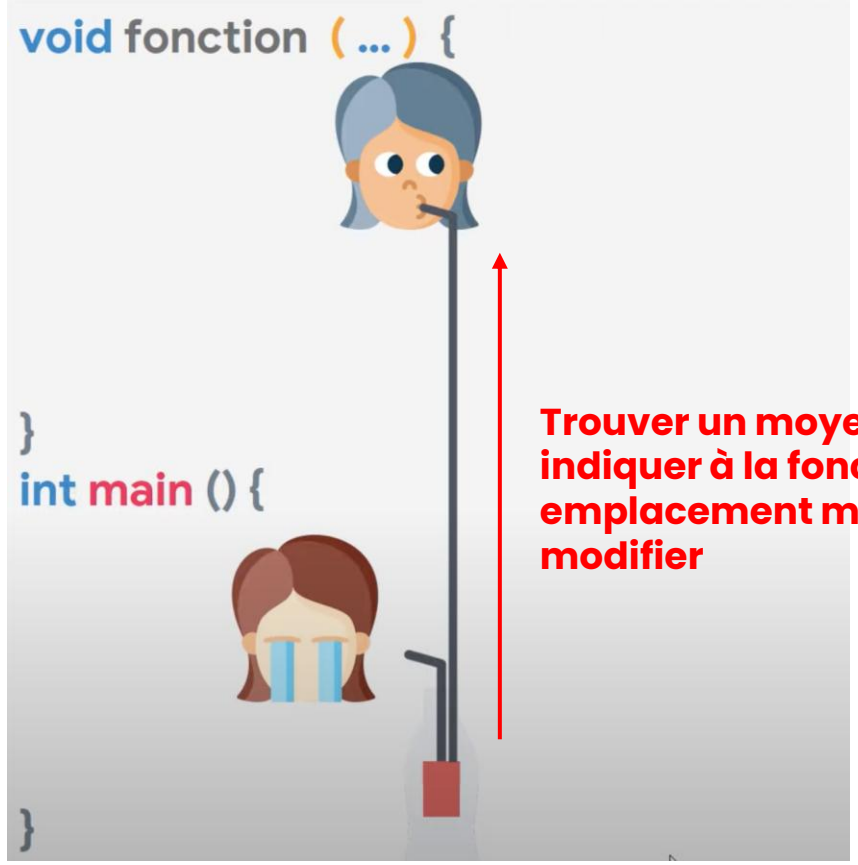
```
Saisir une valeur : 3
Saisir une autre valeur : 4
Avant l'appel de la fonction : x = 3, y = 4
Après l'appel de la fonction : x = 3, y = 4
```

Les valeurs de **x** et **y** n'ont pas changé et il n'est pas possible de retourner plus d'un résultat.

Comment faire ?

Passage des paramètres

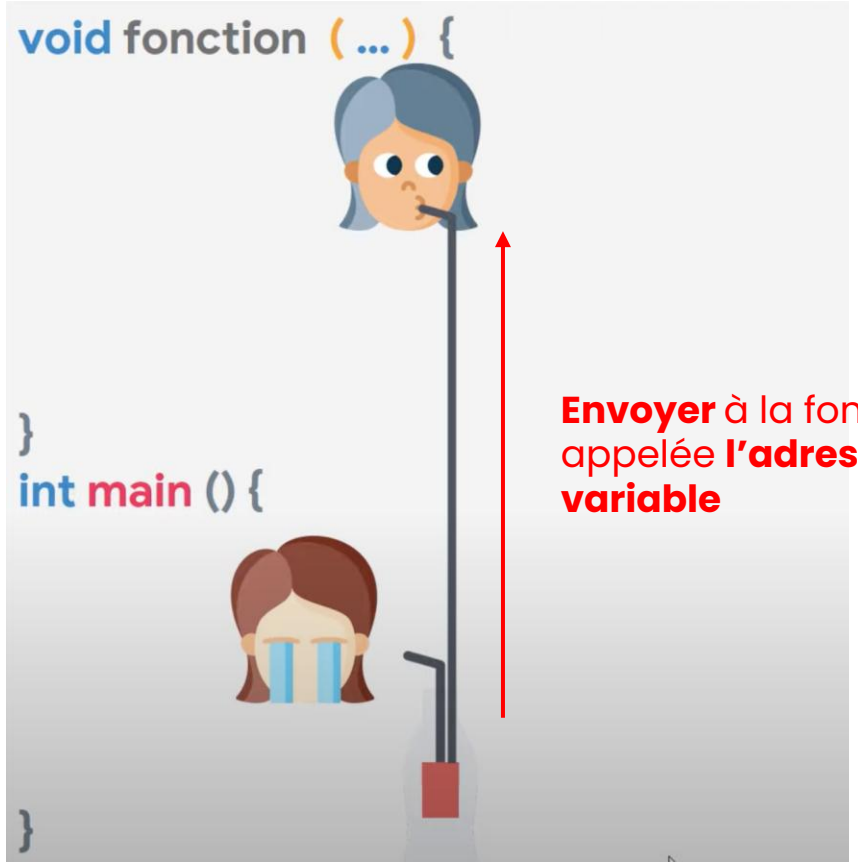
Comment faire pour qu'une fonction modifie une variable du programme appelant ?



Trouver un moyen pour indiquer à la fonction quel emplacement mémoire modifier

Passage des paramètres

Utilisation des pointeurs



- Passer un pointeur (adresse d'une variable) en paramètre d'une fonction permet à cette dernière d'accéder à l'emplacement mémoire de la variable du programme appelant
- Passer un pointeur en paramètre d'une fonction, crée une **copie de l'adresse** dans une variable locale de la fonction.
- Dans le corps de la fonction, le **déréférencement** de l'adresse permet d' accéder à la variable du programme appelant.
- La fonction appelée est alors « en mesure » de **modifier la variable** du programme appelant.

Passage des paramètres

Utilisation des pointeurs

Comment le faire en langage C ?

- Les paramètres de la fonction appelée doivent être déclarés comme des pointeurs ➔ utilisation de l'opérateur *****

- À l'appel de la fonction, mettre les adresses dans les arguments au lieu des valeurs ➔ utilisation de l'opérateur **&**

Passage des paramètres

Utilisation des pointeurs pour modifier des variables par une fonction

Exemple 2

```c
#include <stdio.h>
void permutation(int *x, int *y)
{
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
int main ()
{
    int x, y;
    printf("Saisir une valeur : ");
    scanf("%d", &x);
    printf("Saisir une autre valeur : ");
    scanf("%d", &y);
    printf("Avant l'appel de la fonction : x = %d, y = %d\n", x, y);
    permutation(&x, &y);
    printf("Après l'appel de la fonction : x = %d, y = %d\n", x, y);
    return 0;
}
```

```
Saisir une valeur : 3
Saisir une autre valeur : 4
Avant l'appel de la fonction : x = 3, y = 4
Après l'appel de la fonction : x = 4, y = 3
```

Passage des paramètres

Et pour les tableaux ?

Passage d'un tableau en paramètres

- Étant donné que le nom d'un tableau est lui-même l'adresse de son premier élément, c'est donc celle-ci qui est passée en paramètres.

- Les deux écritures **type * nom_tableau** ou **type nom_tableau[]** sont autorisées dans les paramètres d'une fonction.

- Pour manipuler le tableau dans le corps de la fonction, les deux écritures ***(nom_tableau + i)** et **nom_tableau[i]** sont autorisées.

- À l'appel de la fonction, mettre seulement le nom du tableau comme argument (**sans l'&**).

03

La notion de module

03

La notion de module en C

Définition

- Le découpage d'un projet C en modules est similaire à l'idée de son découpage en fonctions.
- L'objectif est que chaque module corresponde à un tout signifiant.
- Un module est un couple de :
  - Fichier source (extension .c)
  - Fichier entête (« header », extension .h).
- Un module regroupe des fonctions et/ou variables et/ou types traitant **une sous partie** du **problème global**.

La notion de module en C

Fichier entête (header(.h))

- Un fichier source (.c) dans un module est toujours accompagné d'un fichier entête (.h).
- Un fichier entête (.h) contient :
  - Les définitions des types publics
  - Les inclusions des librairies standards utilisées par les fonctions du module
  - Les déclarations des variables globales et constantes
  - Les prototypes des fonctions utilisées dans le module.

La notion de module en C

Fichier source (.c)

- Un fichier source (.c) contient
  - Les définitions des fonctions utilisées dans le module.
  - L'inclusion du fichier (.h) associé

#include « nom_module.h »

Pour un module utilisateur, utiliser les **guillemets** au lieu des < >

La notion de module en C

Finalement, un projet C doit contenir au minimum 3 fichiers :

main.c

```c
#include «fonctions.h»
#include <stdio.h>

int main ()
{
    int x, y, res;
    scanf("%d", &x);
    scanf("%d", &y);
    res = somme(x, y);
    return 0;
}
```

Programme principal: Appel des fonctions

fonctions.h

```c
int somme (int a, int b);
```

Fichier entête contenant la déclaration des prototypes des fonctions, la déclaration des types

fonctions.c

```c
#include «fonctions.h»
int somme (int a, int b)
{
    int resultat;
    resultat = a + b;
    return resultat;
}
```

Fichier où les fonctions sont définies